

# Metody Obliczeniowe w Nauce i Technice

## Laboratorium 10 – Równania różniczkowe zwyczajne, część II

Mateusz Stoch

20 maja 2026

### 1 Zadanie 1(a) – Implementacja metod numerycznych

**Treść:** Dany jest układ równań ruchu ciała w polu grawitacyjnym dwóch mas (CR3BP) w układzie synodycznym:

$$\begin{aligned}x' &= u \\u' &= x + 2v - \left[ \frac{(1 - \mu)(x + \mu)}{r_1^3} + \frac{\mu(x - 1 + \mu)}{r_2^3} \right] \\y' &= v \\v' &= y - 2u - \left[ \frac{(1 - \mu)y}{r_1^3} + \frac{\mu y}{r_2^3} \right]\end{aligned}$$

gdzie  $r_1 = \sqrt{(x + \mu)^2 + y^2}$ ,  $r_2 = \sqrt{(x - 1 + \mu)^2 + y^2}$ ,  $\mu = 0,012150582$ , stan początkowy  $(x, y, u, v) = (1,05; 0,1; -0,45; -0,25)$ , czas symulacji  $t \in [0, 5]$ .

Rozwiąż powyższy układ równań jawną metodą Eulera, niejawną metodą Eulera, półjawną metodą Eulera oraz metodą Rungego-Kutty czwartego rzędu (RK4).

#### 1.1 Jawna metoda Eulera

Jawna metoda Eulera wyznacza pochodną w bieżącym punkcie i przesuwa rozwiązanie o krok  $h$  do przodu:

$$y_{k+1} = y_k + h \cdot f(t_k, y_k)$$

```
1 def euler_explicit(f, t_span, y0, h):
2     t0, tf = t_span
3     t = np.arange(t0, tf + h/2, h, dtype=np.float64)
4     y = np.zeros((len(t), len(y0)), dtype=np.float64)
5     y[0] = y0
6     for k in range(len(t) - 1):
7         y[k+1] = y[k] + h * f(t[k], y[k])
8     return t, y
```

Listing 1: Implementacja jawnej metody Eulera

Metoda jest prosta w implementacji, ale ma rząd zbieżności tylko  $\mathcal{O}(h)$  i jest znana z tendencji do narastania błędu (niestabilność numeryczna dla układów sztywnych lub przy zbyt dużym kroku).

## 1.2 Niejawna metoda Eulera

Niejawna metoda Eulera korzysta z pochodnej wyznaczonej w kroku *następnym*, co wymaga rozwiązania równania:

$$y_{k+1} = y_k + h \cdot f(t_{k+1}, y_{k+1})$$

Ponieważ  $y_{k+1}$  pojawia się po obu stronach równania, rozwiązanie wyznaczono iteracją prostą z tolerancją  $10^{-12}$ :

```
1 def euler_implicit(f, t_span, y0, h, mu=MU):
2     t0, tf = t_span
3     t = np.arange(t0, tf + h/2, h, dtype=np.float64)
4     y = np.zeros((len(t), len(y0)), dtype=np.float64)
5     y[0] = y0
6     for k in range(len(t) - 1):
7         y_guess = y[k].copy()
8         for _ in range(50):
9             y_new = y[k] + h * f(t[k+1], y_guess)
10            if np.linalg.norm(y_new - y_guess) < 1e-12:
11                break
12            y_guess = y_new
13        y[k+1] = y_new
14    return t, y
```

Listing 2: Implementacja niejawnej metody Eulera (iteracja prosta)

Metoda niejawna jest bezwarunkowo stabilna, jednak iteracja prosta może nie zbiegać dla wszystkich układów i dużych kroków, a koszt obliczeniowy jest wyższy niż metody jawnej.

## 1.3 Półjawna (symplektyczna) metoda Eulera

Półjawna metoda Eulera to kompromis między metodą jawną a niejawną. Prędkości są aktualizowane jawnie, natomiast pozycje wyznaczane są przy użyciu już zaktualizowanych prędkości:

```
1 def euler_symplectic(f, t_span, y0, h, mu=MU):
2     t0, tf = t_span
3     t = np.arange(t0, tf + h/2, h, dtype=np.float64)
4     y = np.zeros((len(t), len(y0)), dtype=np.float64)
5     y[0] = y0
6     for k in range(len(t) - 1):
7         f_k = f(t[k], y[k])
8         # Krok 1: zaktualizuj predkosci (u, v) jawnie
9         y_mid = y[k].copy()
10        y_mid[2] += h * f_k[2]
11        y_mid[3] += h * f_k[3]
12        # Krok 2: zaktualizuj pozycje (x, y) uzywajac nowych predkosci
13        f_mid = f(t[k], y_mid)
14        y[k+1] = y[k].copy()
15        y[k+1][0] += h * f_mid[0]
16        y[k+1][1] += h * f_mid[1]
```

```

17         y[k+1][2] = y_mid[2]
18         y[k+1][3] = y_mid[3]
19     return t, y

```

Listing 3: Implementacja półjawnej (symplektycznej) metody Eulera

## 1.4 Metoda Rungego-Kutty 4. rzędu (RK4)

RK4 to klasyczna, szeroko stosowana metoda całkowania ODE o rzędzie zbieżności  $\mathcal{O}(h^4)$ :

$$y_{k+1} = y_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = f(t_k, y_k), \quad k_2 = f\left(t_k + \frac{h}{2}, y_k + \frac{h}{2}k_1\right), \quad k_3 = f\left(t_k + \frac{h}{2}, y_k + \frac{h}{2}k_2\right), \quad k_4 = f(t_k + h, y_k + hk_3)$$

```

1 def rk4(f, t_span, y0, h):
2     t0, tf = t_span
3     t = np.arange(t0, tf + h/2, h, dtype=np.float64)
4     y = np.zeros((len(t), len(y0)), dtype=np.float64)
5     y[0] = y0
6     for k in range(len(t) - 1):
7         k1 = f(t[k], y[k])
8         k2 = f(t[k] + h/2, y[k] + h * k1 / 2)
9         k3 = f(t[k] + h/2, y[k] + h * k2 / 2)
10        k4 = f(t[k] + h, y[k] + h * k3)
11        y[k+1] = y[k] + (h/6) * (k1 + 2 * k2 + 2 * k3 + k4)
12    return t, y

```

Listing 4: Implementacja metody RK4

## 1.5 Porównanie stanów końcowych

Wszystkie metody uruchomiono z krokiem  $h = 0,001$  na przedziale  $t \in [0, 5]$ , co daje 5001 kroków. Stany końcowe:

| Metoda              | $x(5)$  | $y(5)$  |
|---------------------|---------|---------|
| Euler jawny         | 0,4613  | 0,2499  |
| Euler niejawny      | -0,0614 | -0,1007 |
| Euler symplektyczny | -0,0690 | 0,8083  |
| RK4                 | 0,0100  | 0,8767  |

Różnice w stanach końcowych między metodami są wyraźne, co wynika zarówno z różnych rzędów dokładności, jak i różnych właściwości stabilności każdej metody.

## 2 Zadanie 1(b) – Mapa układu

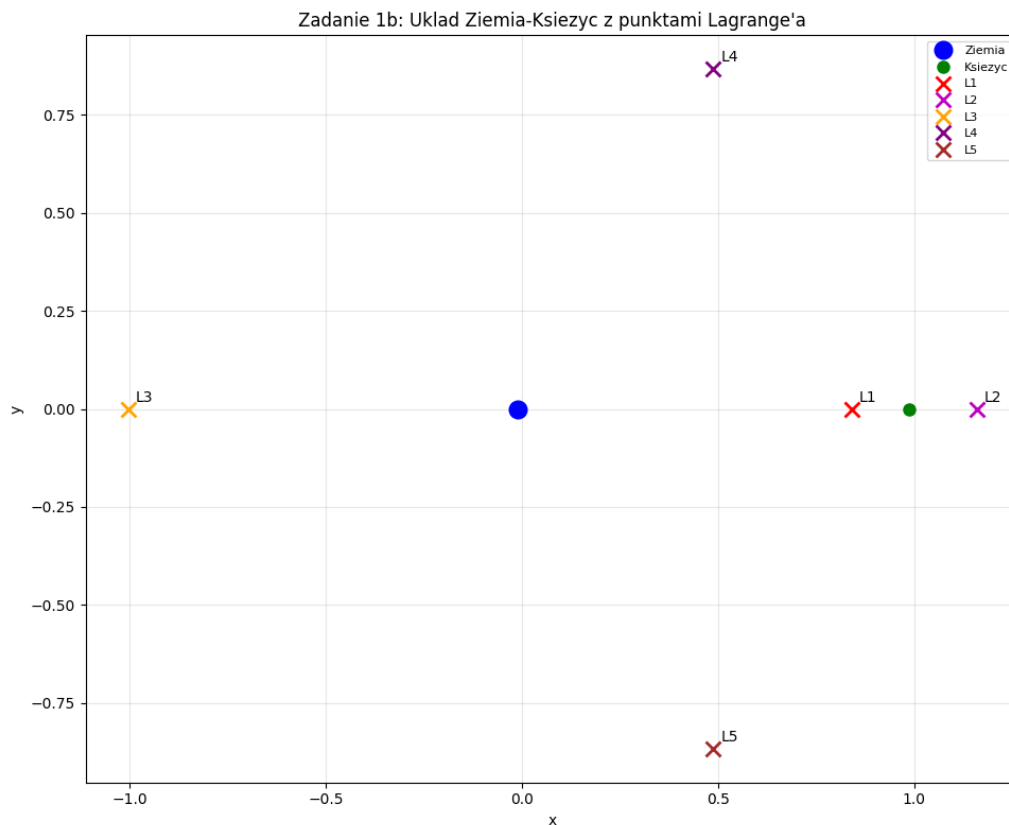
**Treść:** Zaznacz położenie obu mas oraz położenie punktów libracyjnych Lagrange’a L1–L5, wyznaczonych ze wzorów:

$$L_1 = \left(1 - \sqrt[3]{\frac{\mu}{3}}, 0\right), \quad L_2 = \left(1 + \sqrt[3]{\frac{\mu}{3}}, 0\right), \quad L_3 = \left(-\left(1 + \frac{5\mu}{12}\right), 0\right)$$

$$L_4 = \left(\frac{1}{2} - \mu, \frac{\sqrt{3}}{2}\right), \quad L_5 = \left(\frac{1}{2} - \mu, -\frac{\sqrt{3}}{2}\right)$$

Obliczone współrzędne:

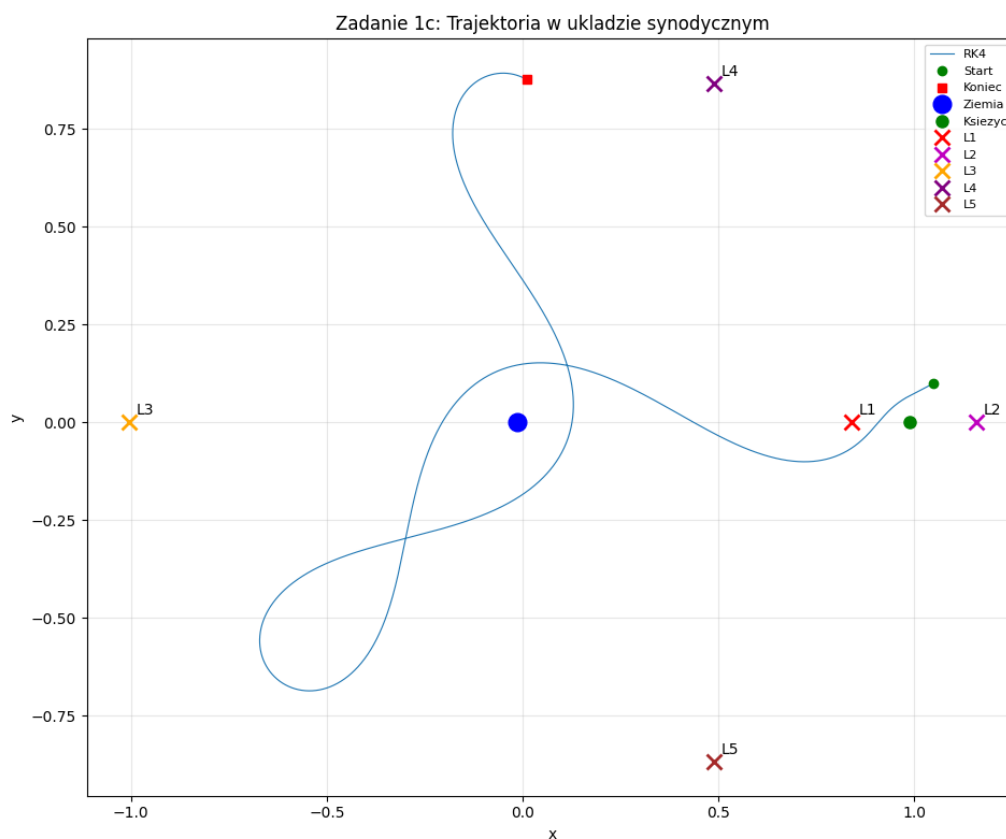
| Punkt | $x$       | $y$       |
|-------|-----------|-----------|
| L1    | 0,840599  | 0,000000  |
| L2    | 1,159401  | 0,000000  |
| L3    | -1,005063 | 0,000000  |
| L4    | 0,487849  | 0,866025  |
| L5    | 0,487849  | -0,866025 |



Wykres 1: Mapa układu: położenia obu mas i pięciu punktów libracyjnych Lagrange'a (L1, L2, L3, L4, L5) w układzie synodycznym.

### 3 Zadanie 1(c) – Trajektoria w układzie synodycznym

**Treść:** Narysuj trajektorię ruchu ciała  $(x(t), y(t))$  w układzie synodycznym (obracającym się razem z układem dwóch mas).



Wykres 2: Trajektoria ruchu ciała wyznaczona metodą RK4 w układzie synodycznym.

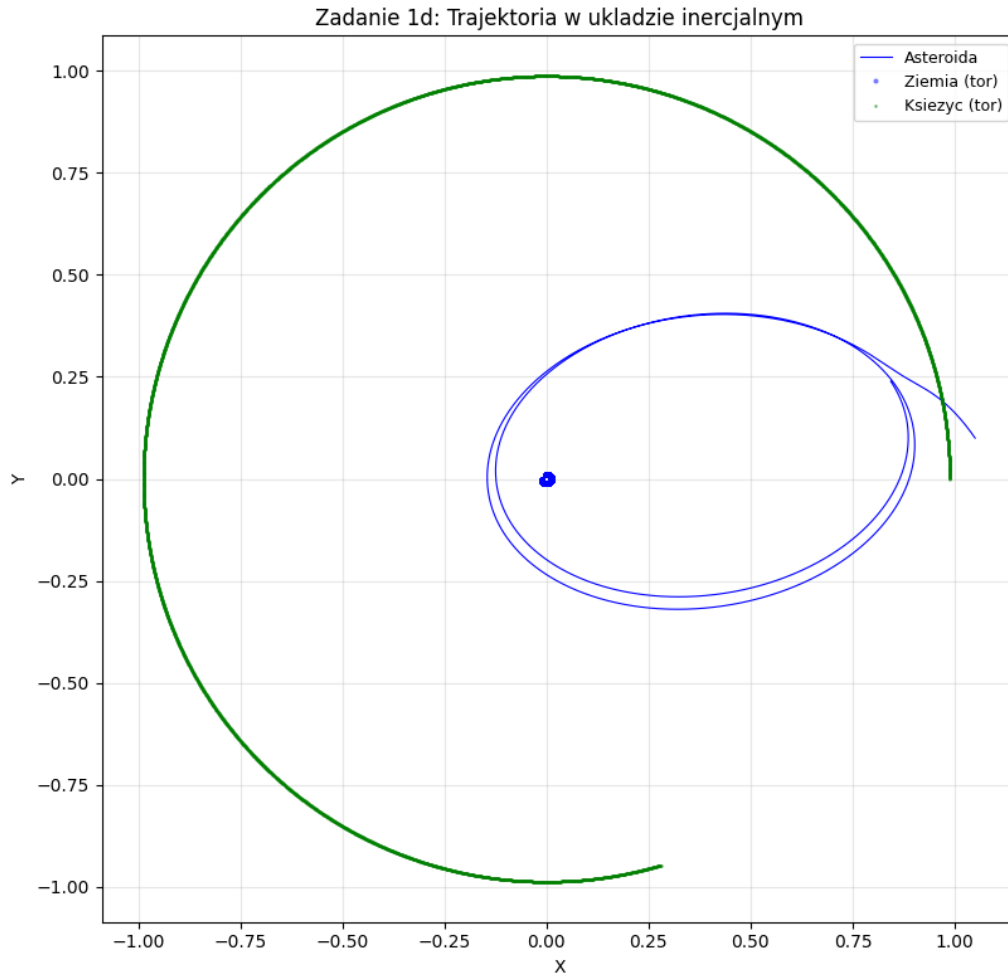
Trajektoria w układzie synodycznym jest złożona i nie przypomina prostego eliptycznego toru.

## 4 Zadanie 1(d) – Trajektoria w układzie inercyjnym

**Treść:** Narysuj trajektorie ruchu ciała w nieruchomym układzie inercyjnym, korzystając z transformacji obrotowej:

$$\begin{pmatrix} X \\ Y \end{pmatrix} = \begin{pmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Na jaką najbliższą odległość ciało zbliży się do masy  $m_1$ ?



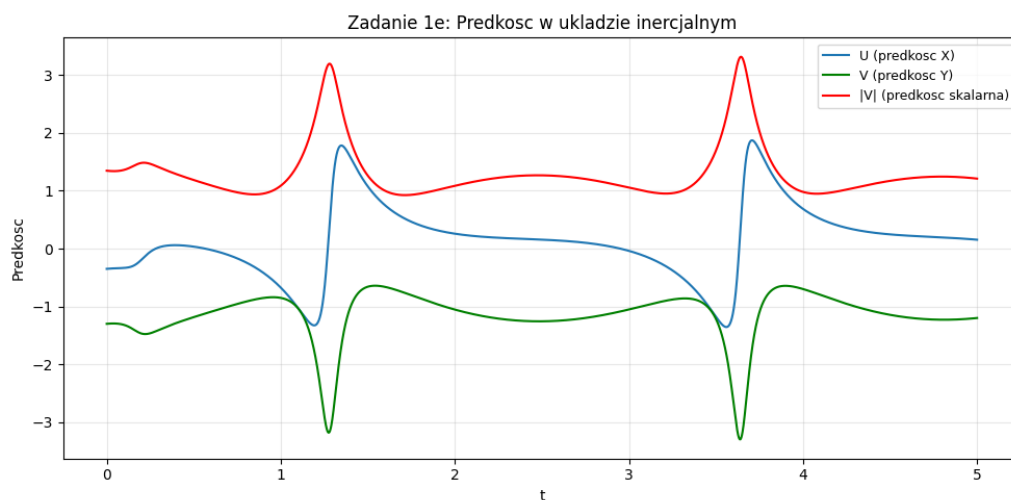
Wykres 3: Trajektorie ruchu ciał w nieruchomym układzie inercyjnym (po transformacji obrotowej). Zaznaczono też toru ruchu obu mas.

W układzie inercyjnym obie masy poruszają się po okręgach wokół wspólnego środka masy. Badane ciało zbliży się na odległość 0.134530 j.u.

## 5 Zadanie 1(e) – Prędkość w układzie inercyjnym

**Treść:** Narysuj wykres prędkości ciała w układzie inercyjnym, korzystając z transformacji:

$$\begin{pmatrix} U \\ V \end{pmatrix} = \begin{pmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} + \begin{pmatrix} \sin t & \cos t \\ -\cos t & \sin t \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

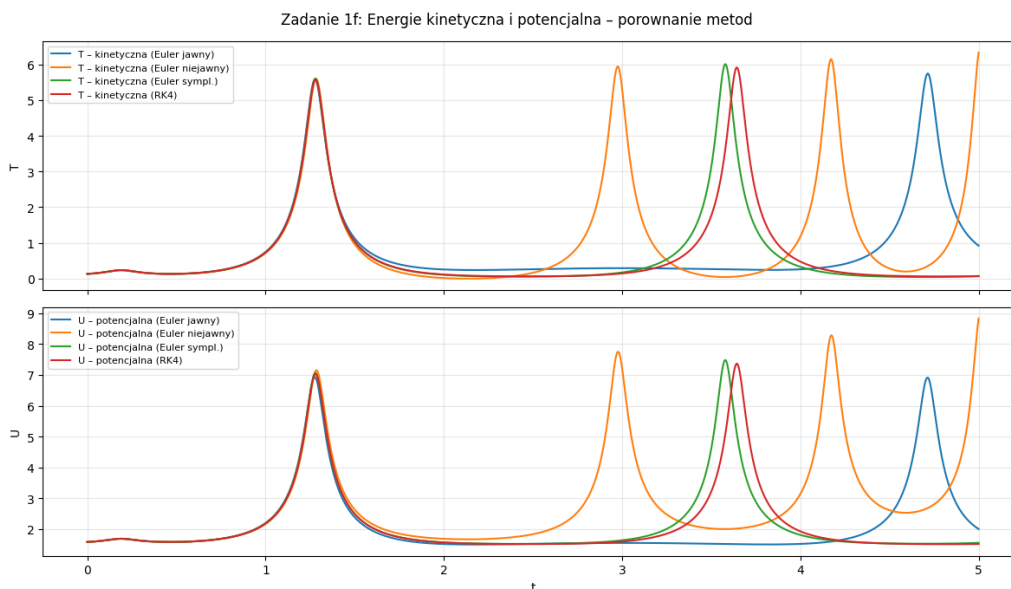


Wykres 4: Składowe prędkości ( $U, V$ ) badanego ciała w układzie inercyjnym w funkcji czasu, dla wszystkich czterech metod.

## 6 Zadanie 1(f) – Energia kinetyczna i potencjalna

**Treść:** Narysuj wykresy energii kinetycznej  $T$  oraz energii potencjalnej  $U$  ciała w funkcji czasu:

$$T = \frac{1}{2}(u^2 + v^2), \quad U = \frac{1}{2}(x^2 + y^2) + \frac{1-\mu}{r_1} + \frac{\mu}{r_2}$$



Wykres 5: Energia kinetyczna  $T$  i potencjalna  $U$  badanego ciała w funkcji czasu, dla wszystkich czterech metod.

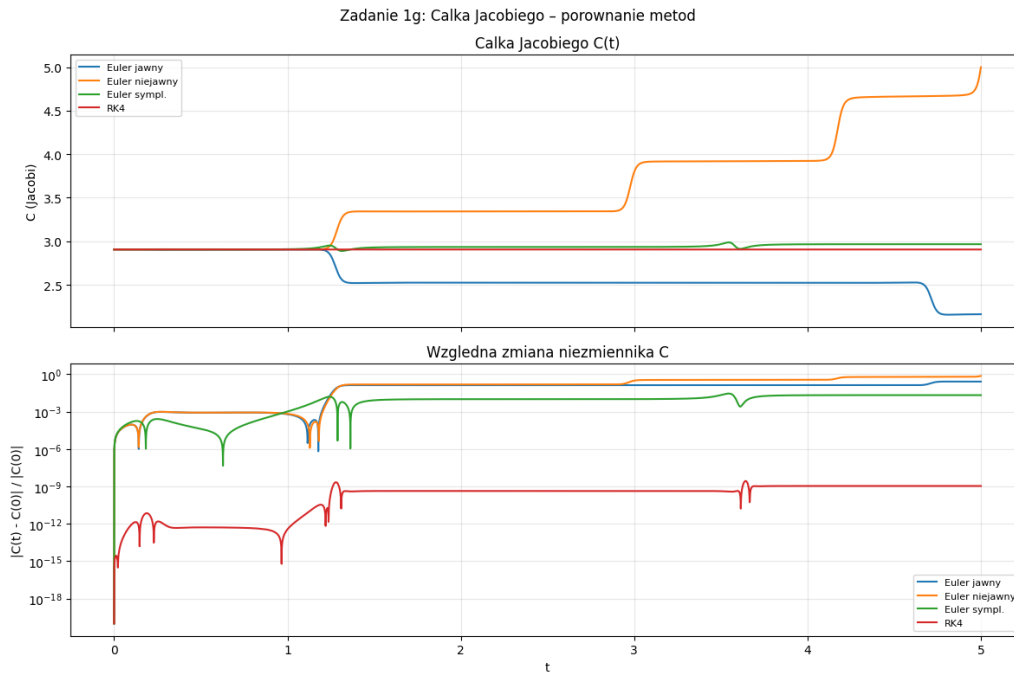
Wykres 5 pokazuje, że różne metody dają wyraźnie różniące się przebiegi energii, co bezpośrednio wynika z różnych błędów popełnianych przez każdy schemat całkowania.

## 7 Zadanie 1(g) – Całka Jacobiego

**Treść:** Sprawdź, czy otrzymane numerycznie rozwiązania zachowują niezmiennik ruchu, jakim jest całka Jacobiego  $C$ :

$$C = x^2 + y^2 + \frac{2(1-\mu)}{r_1} + \frac{2\mu}{r_2} - u^2 - v^2 = \text{const}$$

Narysuj wykres wartości  $C$  w funkcji czasu dla każdej z metod.



Wykres 6: Wartość całki Jacobiego  $C$  w funkcji czasu dla wszystkich czterech metod.

Analiza Wykresu 6 pozwala ocenić jakość poszczególnych schematów numerycznych:

- **Euler jawny** wyraźnie *narusza* stałość całki Jacobiego. Wartość  $C$  stopniowo rośnie wraz z czasem, co świadczy o systematycznym narastaniu błędu energetycznego.
- **Euler niejawnym** wykazuje odwrotną tendencję, tzn. wartość  $C$  maleje, metoda pochłania energię.
- **Euler symplektyczny** zachowuje całkę Jacobiego znacznie lepiej niż oba poprzednie warianty. Fluktuacje  $C$  są małe i ograniczone, bez trendu systematycznego.
- **RK4** daje bardzo małe odchylenia od wartości początkowej  $C$  (rzędu błędów maszynowych dla tego kroku), co potwierdza wysoki rząd dokładności tej metody.

## 8 Wnioski

1. **Euler jawny** jest najprostszy w implementacji, lecz systematycznie narusza całkę Jacobiego. Przy zbyt dużym kroku lub długich symulacjach traci dokładność i może stać się niestabilny.

2. **Euler niejawny** jest bezwarunkowo stabilny numerycznie, jednak wartość całki Jacobiego maleje w czasie. Dla układów zachowawczych jest to niekorzystna właściwość. Ponadto wymaga rozwiązywania równania nieliniowego w każdym kroku.
3. **Euler symplektyczny** stanowi dobry kompromis. Brak trendu w całce Jacobiego, wartość  $C$  oscyluje wokół wartości początkowej bez długookresowych odchyień. Koszt obliczeniowy jest porównywalny z metodą jawną.
4. **RK4** oferuje wysoki rząd zbieżności ( $\mathcal{O}(h^4)$ ) i bardzo dobre zachowanie całki Jacobiego przy umiarkowanym kroku. Jest to metoda ogólnego przeznaczenia, doskonale sprawdzająca się w praktyce.